UNIVERSIDAD GRABRIEL RENE MORENO

FACULTAD DE INGENIERIA EN CIENCIAS DE LA COMPUTACION Y TELECOMUNICACIONES

ESTRUCTURA DE DATOS II

TAREA 4. Lista de Arboles

ESTADO: Terminado (100%)

INTEGRANTES	REGISTRO
Benitez Arroyo Joseph	221043837
Montaño Claros Wilson	220001911
Navarro Valle Robert Lorenzo	219036489
Torrez Vaca Andres	220153914
Vino Apaza Vanessa	220053243

COMENTARIO:

Se realizo con éxito la Tarea 4, lo más interesante de esta actividad fue el procesamiento de Archivos donde el primer árbol creado se guardaba tomando en cuenta la longitud y en el segundo árbol se guardaba tomando en cuenta la frecuencia con la que aparecen las palabras en dicho archivo, se realizó un ejemplo con “The Call of Cthulhu” donde la palabra más larga tiene 19 caracteres y la palabra con mayor frecuencia es “the” que se repite 807 veces.

Fecha de presentación: Domingo, 12 de Noviembre de 2023

Fecha Presentada: Sábado, 11 de Noviembre de 2023

Días de Retraso: 0

SANTA CRUZ – BOLIVIA

Para los siguientes ejercicios, leer los datos de un archivo de texto y encontrar la salida en consola y en otro archivo de texto de salida.

```
public class Nodo {
    public Nodo izq;
    public Nodo der;
    public String str;
    public int frec;

    public Nodo(String str){
        this.str=str;
        izq=der=null;
    }

    public Nodo(String x, int frecc){
        this.str=x;
        if(frecc!=0)
            this.frec=frecc;
        else
            this.frec=1;
        this.izq=this.der=null;
    }
}

public class Arbol {
    public Nodo raiz;

    public Arbol(){
        raiz=null;
    }
```

```
//metodos de insercion
public void insertar(String str){
    raiz=insertar(raiz, str);
}

private Nodo insertar(Nodo p, String str){
    if(p==null)
        return new Nodo(str);

      if(str.equals(p.str)){
        p.frec++;
        return p;
    }

    if(str.compareTo(p.str)<0)
        p.izq=insertar(p.izq, str);
    else
        p.der=insertar(p.der,str);

    return p;
}

public void insertar(String str, int frec){
    raiz=insertar(raiz, str, frec);
}
```

```
private Nodo insertar(Nodo p, String cad, int frec){
    if(p==null)
        return new Nodo(cad, frec);

   if(CompareMenor(p.str, cad))
        p.der=insertar(p.der, cad, frec);
    else
        p.izq=insertar(p.izq,cad, frec);

    return p;
}

private boolean CompareMenor(String cad, String canto){
    if(cad.length()>canto.length())
        return false;

    if(cad.length()<canto.length())
        return true;

    if(cad.length()==canto.length()){
        for (int i = 0; i < cad.length(); i++) {
            if(cad.charAt(i)<canto.charAt(i))
                return true;

            if(cad.charAt(i)>canto.charAt(i))
                return false;
        }
    }
    return true;
```

```
}

//Ordenamientos
//1
public void inOrden(){
    inOrden(raiz);
}

private void inOrden(Nodo p){
    if(p==null)
        return ;

    inOrden(p.izq);
    System.out.print("["+p.str+"]");
    inOrden(p.der);
}
//2
public void inOrdenRev(){
    inOrdenRev(raiz);
}

public void inOrdenRev(Nodo p){
    if(p==null)
        return ;

    inOrdenRev(p.der);
    System.out.print("["+p.str+"]");
    inOrdenRev(p.izq);
}
```

```
    }
```

A. Dado una secuencia de n-palabras ingresados. Mostrar las palabras de menor a mayor por longitudes de palabras.

Para resolver este problema, utilizar una Lista de ABB. Implementar los siguientes métodos:

```
public class ArrayTree {
    public int max;
    public Arbol arbol[];

    public ArrayTree(int max){
        this.max=max;
        this.arbol=new Arbol[max];
        for (int i = 0; i < max; i++) {
            arbol[i]=new Arbol();
        }
    }

    public void insertar(String str){//asc asc
        arbol[str.length()].insertar(str);
    }

   public void inOrden(){
        for (int i = 0; i < max; i++) {
            arbol[i].inOrden();
            if(arbol[i].raiz!=null)
                System.out.println();
        }
    }
```

- **L1.palabrasAscAsc() : Método que muestra las palabras ordenados de menor a mayor por longitud de palabra de menor a mayor.**

```
public void palabrasAscAsc(){
    for (int i = 0; i < max; i++) {
        arbol[i].inOrden();
        if(arbol[i].raiz!=null)
            System.out.println();
    }
}
```

- **L1.palabrasAscDesc() : Método que muestra las palabras ordenados de menor a mayor por longitud de palabra de mayor a menor.**

```
public void palabrasAscDes(){
    for (int i = 0; i < max; i++) {
        arbol[i].inOrdenRev();
        if(arbol[i].raiz!=null)
            System.out.println();
    }
}
```

- **L1.palabrasDescAsc() : Método que muestra las palabras ordenados de mayor a menor por longitud de palabra de menor a mayor.**

```
public void palabrasDesAsc(){
    for (int i = max-1; i > 0; i--) {
        arbol[i].inOrden();
        if(arbol[i].raiz!=null)
            System.out.println();
    }
```

```
}
```

- **L1.palabrasDescDesc() : Método que muestra las palabras ordenados de mayor a menor por longitud de palabra de mayor a menor.**

```
public void palabrasDesDes(){
    for (int i = max-1; i > 0; i--) {
        arbol[i].inOrdenRev();
        if(arbol[i].raiz!=null)
            System.out.println();
    }
}
```

B. Dado una secuencia de n-palabras. Encontrar la frecuencia de cada palabra.

```
public class ArrayTreeFrec {
    public int max;
    public Arbol arbol[];

    public ArrayTreeFrec(int max){
        this.max=max;
        this.arbol=new Arbol[max];
        for (int i = 0; i < max; i++) {
            arbol[i]=new Arbol();
        }
    }

    public void insertar(String str, int frec){//asc asc
        arbol[frec].insertar(str, frec);
    }
```

```
public void arboleda(ArrayTree arr){
    for (int i = 0; i < arr.max; i++) {
        if(arr.arbol[i].raiz!=null){
            recorrido(arr.arbol[i]);
        }
    }
}

public void recorrido(Arbol ar){
    recorrido(ar.raiz);
}

private void recorrido(Nodo p){
    if(p==null)
        return;

    insertar(p.str, p.frec);
    recorrido(p.izq);
    recorrido(p.der);
}
```

- **A1.menorMayor() : Método que muestra las palabras de menor a mayor con sus respectivas frecuencias.**

```
public void menorMayor(){
    for (int i = 0; i < max; i++) {
        if(arbol[i].raiz!=null){
            System.out.println();
            System.out.print("Frecuencia: "+(i+1)+".-");
```

```
            arbol[i].inOrden();
        }
    }
  }
```

- **A1.mayorMenor(): Método que muestra las palabras de mayor a menor con sus respectivas frecuencias.**

```
  public void mayorMenor(){
      for (int i = 0; i < max; i++) {
          if(arbol[i].raiz!=null){
              System.out.println();
              System.out.print("Frecuencia: "+(i+1)+".-");
              arbol[i].inOrdenRev();
          }
      }
  }
}
```

Elaborar un informe sobre un Análisis de Contenido, sobre las Palabras de un Archivo de Texto Extenso. Adjuntar informe y archivos de texto.

//Procesamiento de Archivos

```
class ArrayTree {
  public void tokenizer(String cad){
      String[] tokens=cad.split("\\s+");
      for (String token : tokens) {
          insertar(token.toLowerCase());
      }
  }
```

```
    public void convert(String ruta){

      try {
          File file = new File(ruta);
          FileReader fileReader = new FileReader(file);
          BufferedReader bufferedReader = new BufferedReader(fileReader);
          String line;
          while ((line = bufferedReader.readLine()) != null) {
              tokenizer(line);
          }
          bufferedReader.close();
      } catch (IOException e) {
          e.printStackTrace();
      }
    }
}
```

//Archivo de Texto de Salida en el Main

```
    public static void main(String[] args) {
      //System.out.println("Hello World!");

      ArrayTree l1= new ArrayTree(20);
      ArrayTreeFrec ar=new ArrayTreeFrec(808);

      PrintStream console = System.out;

      try {
          String rutaArchivo = "C:\\Users\\Benitez\\Desktop\\output.txt";
          File file = new File(rutaArchivo);
```

```
FileOutputStream fos = new FileOutputStream(file);
PrintStream ps = new PrintStream(fos);
System.setOut(ps);

l1.convert("C:\\Users\\Benitez\\Downloads\\The Call of Cthulhu.txt");
ar.arboleda(l1);
System.out.println("ArrayTree palabrasAscAsc");
l1.palabrasAscAsc();
System.out.println();
System.out.println();
System.out.println("ArrayTreeFrec menorMayor");
ar.menorMayor();
System.out.println();
System.out.println();
System.out.println("ArrayTreeFrec mayorMenor");
ar.mayorMenor();
System.out.println();
System.out.println();
System.out.println("ArrayTree palabrasAscDes");
l1.palabrasAscDes();
System.out.println();
System.out.println();
System.out.println("ArrayTree palabrasDesAsc");
l1.palabrasDesAsc();
System.out.println();
System.out.println();
System.out.println("ArrayTree palabrasDesDes");
l1.palabrasDesDes();
System.setOut(console);
```

```
        fos.close();

        System.out.println("ArrayTree palabrasAscAsc");
        l1.palabrasAscAsc();
        System.out.println();
        System.out.println();
        System.out.println("ArrayTreeFrec menorMayor");
        ar.menorMayor();
        System.out.println();
        System.out.println();
        System.out.println("ArrayTreeFrec mayorMenor");
        ar.mayorMenor();
        System.out.println();
        System.out.println();
        System.out.println("ArrayTree palabrasAscDes");
        l1.palabrasAscDes();
        System.out.println();
        System.out.println();
        System.out.println("ArrayTree palabrasDesAsc");
        l1.palabrasDesAsc();
        System.out.println();
        System.out.println();
        System.out.println("ArrayTree palabrasDesDes");
        l1.palabrasDesDes();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```